

1 DMC tutorial - How to create a plug-in

1.1 Overview

This document shows how to create a plug-in for DMC using a simple example. You will learn the structure of a plug-in, how it is embedded and configured, how to debug a plug-in, and then you are able to integrate your systems into DMC using their custom plug-ins.

The source code of a finished plug-in is located in the DMC-SDK sample CreatePlugin.

The plug-in should offer the following functions:

- After an external pulse, an event is sent with a message.
- This message can be configured.
- The plug-in can be easily integrated into the DMC and configured as usual.

Four stages are defined to achieve this objective:

1. Create and configure a plug-in project.
2. Create a plug-in that sends a permanently programmed message.
3. Extend the plug-in by configurability of the message.
4. Control of message output by external trigger.

1.2 Create and configure a project

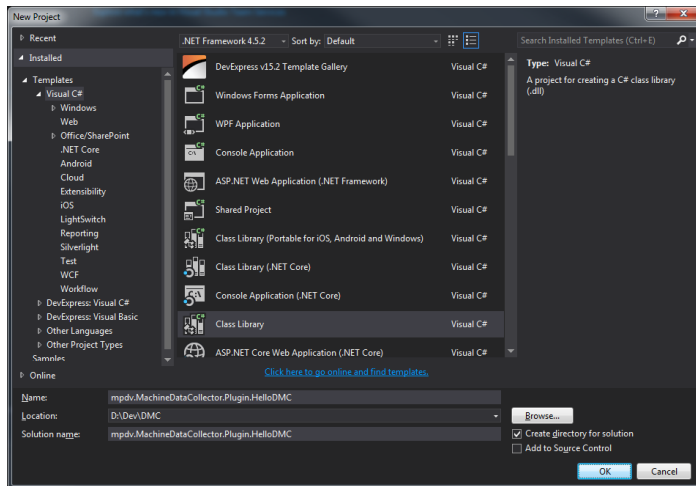
The following file structure is used for this documentation:

```
- DMC
  - bin
    - Plugins
      - Dmc
        - mpdv.MachineDataCollector.Dmc.dll
        - ...
      - ...
    - mpdv.MachineDataCollector.Core.dll
    - mpdv.MachineDataCollector.Gui.exe
    - ...
  - config.xml
  - factory.xml
```

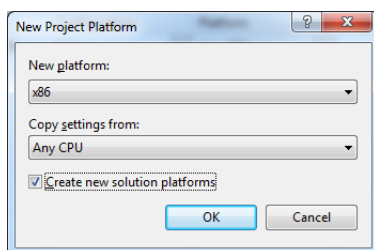
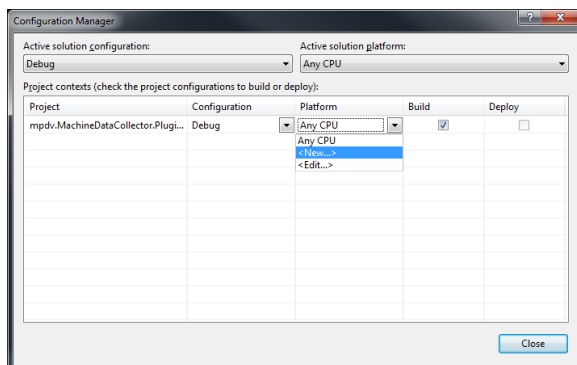
The config.xml as well as factory.xml are taken from the delivery standard. It is only necessary to ensure that the correct reference to the "factory.xml" is entered in the config.xml (originating from the DMC run directory for relative paths):

```
<CollectorConfiguration ... FactoryModel="..\factory.xml">
```

The Visual Studio creates a new project type Class Library. The project has the name `mpdv.MachineDataCollector.Plugin.HelloDMC`. Please ensure that the correct framework is selected (currently .NET 4.5.2). We create the new project in the folder DMC (file structure see above).



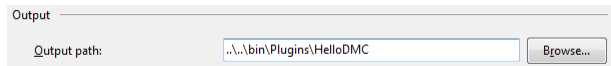
First, the build configuration is changed. To do this, go to Build -> Configuration Manager and create a new configuration for x86.



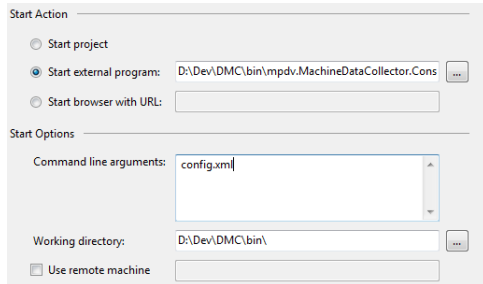
Delete the configuration "Any CPU" because we will not use it.

Then the newly created build configuration has to be changed for debugging (Debug ->mpdv.MachineDataCollector.Plugin.HelloDMC Properties).

The output path is specified to „..\..\bin\Plugins\HelloDMC“ (Build → Output Path).



For debugging (tab Debug), the console is specified as an external startup program. This also requires the server configuration and a *working directory*. (Note that the server configuration is searched for a level above the run directory!).

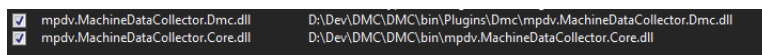


This configuration is saved and, if not already done, the solution configuration *Debug* - x86 is selected as active.

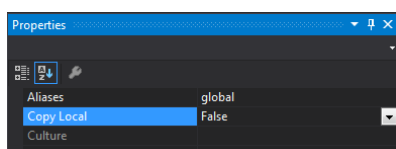
You need two references to develop DMC plug-ins.

- Bin \ mpdv.MachineDataCollector.Core.dll: Provides core functions, such as eventing, capability management, plug-in loading, logging, etc.
- bin \ Plugins \ Dmc \ mpdv.MachineDataCollector.Dmc.dll: Includes DMC-specific logic, such as handling workplaces, orders, operations, and various auxiliary classes.

The references can be added via Project -> Add Reference ... -> Browse.



For *references* to DMC files, CopyLine must be set to *False* (right-click on reference -> Properties -> CopyLocal), otherwise conflicts may occur during dynamic loading.



1.3 Output of a permanently programed message.

A DCM plug-in contains at least one component. For simplicity, however, it is useful to create an entity. This is a class derived from BaseEntity, which in turn inherits from CapableComponent and provides the functionality required.

An entity has capabilities describing its functionality. Capabilities can define parameters and events, where the first are input parameters (configuration or events) and the last are output events (results). For example, a barcode scanner entity can supply the following capabilities:

- Scan
 - Parameters:
 - ScanMode
 - ScanPattern
 - Events:
 - ScanResult

First, we rename the *Class1* created by default into *HelloDMCEntity*. As a base class we use *BaseEntity* and implement the constructor as well as the two abstract functions. Our basic structure now looks as follows:

```
using System.Collections.Generic;
using mpdv.MachineDataCollector.Core.Capabilities;
using mpdv.MachineDataCollector.Core.Data;
using mpdv.MachineDataCollector.Jis.Entities;

namespace mpdv.MachineDataCollector.Plugin.HelloDMC
{
    public class HelloDMCEntity : BaseEntity
    {
        public HelloDMCEntity() : base(new EventType[0], new string[0])
        {
        }

        protected override void Configure(string capabilityType)
        {
        }

        protected override IDictionary<string, Capability> InitPrepareCapabilities()
        {
            return new Dictionary<string, Capability>();
        }
    }
}
```

The constructor of the Baseclass *BaseEntity* expects two arguments:

- **acceptedEventTypes:** The event types that can be received by this class. If the entity does not receive any events, an empty list can be specified here.
- **allowedParameters:** Parameters to configure the entity. Specific configuration parameter. There is no need to specify the configuration of capabilities (see below).

The method *InitPrepareCapabilities()* returns the capabilities supplied by the entity.

The *Configuration ()* method is automatically called for various status changes (for example, changing the current workflow). Here the entity is configured for the next working step according the capability configuration.

Capability is a kind of meta class. Its inputs and outputs, and its parameters are implemented as methods and properties. These methods and properties are decorated with special attributes. A more detailed description of this concept can be found in MBL_DMC_ComponentSystem.

We create a Capability SayHelloCapability, which for the sake of simplicity is implemented as a subclass of HelloDMCEntity.

```
private class SayHelloCapability : BaseCapability
{
    private const string CapabilityType = "SayHello";

    public SayHelloCapability(string id) : base(id, CapabilityType)
    {
    }

    [CapabilityOutput]
    public NamedParams HelloMessage()
    {
        return new NamedParams
        {
            ["HelloMessage"] = "Hello World!"
        };
    }
}
```

The capability is added to the field in order to use it...

```
private SayHelloCapability _sayHelloCapability =
    new SayHelloCapability("F2DBAFCD-C067-412C-B6E1-F153A0527FD0");
```

... and returned in InitPrepareCapabilities().

```
return new Dictionary<string, Capability>
{
    ["SayHello"] = _sayHelloCapability
};
```

To issue the message at the start of the entity, the StartComponent () function is overwritten. These and some other features are requested during the lifetime of all components. The sequence is essential:

- **InitComponent()**: Is called during the initialization but before the start of all components. In InitComponent (), the configuration should be read and the component prepared for the startup. As it is not guaranteed that all components are loaded, it is not yet possible to access other components (Parent / Children or similar).
- **StartComponent()**: Is called during startup for each component. At this point, it is guaranteed that all components are loaded and initialized. Cross-references to other components can be generated where necessary.

- StopComponent(): The component is to be stopped, usually because the application is terminated. At this point, all resources that are used should be released to avoid unexpected behavior or data loss.

The EventFactory generates and sends a message in StartComponent().

```
public override void StartComponent()
{
    base.StartComponent();

    var data = _sayHelloCapability.HelloMessage();
    SendMessage(Core.Engine.EventFactory.CreateEvent(EventType.DmcData, data));
}
```

The configuration is missing and thus, the DMC plug-in cannot be embedded. Therefore, we add the following entry to the factory.xml:

```
<ComponentConfiguration Name="HelloDMCEntity"
    Class="mpdv.MachineDataCollector.Plugin.HelloDMC.HelloDMCEntity" />
```

This entry is used to register the entity and publish under the name HelloDMCEntity. It is essential to specify the complete class name. This is the only way DMC can load and instantiate the entity.

We can now start the DMC server and find an entry in the log file (C: \ ProgramData \ mpdv \ mdc \ config.xml.log):

```
... HelloDMCEntity(JisData)->Global: {"HelloMessage":"Hello World!"}
```

The plug-in was loaded and started and the message was sent as an event. Following, the functionality is extended from this base by configuration and event input.

1.4 Configuration of the message

So far, we have defined only one output event for our capability SayHello. The simplest way to make an entity configurable is to define capability parameters. The capability parameters can be automatically filled with values via the configuration.

Extend the SayHelloCapability as follows:

```
[CapabilityParameter, DefaultValue("Hello World!")]
public string MessageText { get; set; }
```

Assign a default value to the capability parameter. This is loaded in BaseEntity.StartComponent() if no configuration overwrites the default value.

To use the parameter, the method HelloMessage() is extended by a query of the current value.

```
return new NamedParams
{
    ["HelloMessage"] = MessageText
};
```

If DMC is restarted, the log entry appears as before.

You can use the configuration to change the message text. For this purpose, we add the new parameter SayHello_MessageText to the ComponentConfiguration:

```
<ComponentConfiguration Name="HelloDMCEntity"
    Class="mpdv.MachineDataCollector.Plugin.HelloDMC.HelloDMCEntity">
    <Parameter Name="SayHello_MessageText" Value="Hello DMC!" />
</ComponentConfiguration>
```

Each capability parameter can be set in this form via the configuration. The parameter name is composed of [CapabilityType] _ [CapabilityParameter].

If we start DMC again, "Hello DMC!" is issued:

```
... HelloDMCEntity(JisData)->Global: {"HelloMessage":"Hello DMC!"}
```

1.5 To externally control message output

Currently, the message "Hello DMC!" is issued only once when starting the DMC. An event is used to determine when the message is issued. In principle, each event can serve as input to an entity, but it is easier to use a standard component as a transmitter, the TimedTrigger. This component sends a TimerFired event in a configurable interval, which is used to respond with the "Hello DMC!" message.

First, you configure a timer in the *factory.xml*:

```
<ComponentConfiguration Name="HelloDMCTimer"
    Class="mpdv.MachineDataCollector.Core.Components.TimedTrigger">
    <Parameter Name="[SayHello]" Value="5" />
</ComponentConfiguration>
```

This configuration creates a new *TimedTrigger* that sends a TimerFired event named *SayHello* every five seconds. The parenthesis of the parameter name [] is important: a data value is declared by means of a square bracket. In contrast to normal configuration parameters, defined at the implementation time, any data values can be specified. In the case of the TimedTrigger, it is possible to cover various events with different intervals via a trigger.

To receive the event sent by the HelloDMCTimer, it must be added to the list of accepted events in the entity. This is done in the constructor:

```
public HelloDMCEntity() : base(new[] { EventType.TimerFired }, new string[0])
{
}
```

Events are received using the `DeliverMessage ()` function. In this, a distinction can be made between different event types in order to perform corresponding actions. In this case, an individual message is to be sent when the `SayFello` timerFired event is received. To do this, override `DeliverMessage ()` as follows:

```
public override bool DeliverMessage([NotNull] IEvent message)
{
    if (message.Type == EventType.TimerFired &&
        message.TryGetParameter(TimerEvent.TimerName) == "SayHello")
    {
        var values = TryGetCapabilityParameterValues("SayHello");
        var answer = Core.Engine.EventFactory.CreateEvent(EventType.JisData)
            .SetParameter("HelloMessage", values["MessageText"]);
        SendMessage(answer);

        return true;
    }

    return base.DeliverMessage(message);
}
```

Return *true* Because the `TimerFired` event has been treated. Thus, no other entity will receive the event and will respond to it undesirably.

Send the message if requested. Therefore, the `StartComponent ()` method can be removed - it is not required anymore.

Let DMC run for a few seconds, then you will see regular entries in the log file.

```
... 21:52 ... HelloDMCTimer(TimerFired)->Global: {"timer.name":"SayHello"}
... 21:52 ... HelloDMCEntity(JisData)->Global: {"HelloMessage":"Hello World and DMC!"}
... 21:57 ... HelloDMCTimer(TimerFired)->Global: {"timer.name":"SayHello"}
... 21:57 ... HelloDMCEntity(JisData)->Global: {"HelloMessage":"Hello World and DMC!"}
... 22:02 ... HelloDMCTimer(TimerFired)->Global: {"timer.name":"SayHello"}
... 22:02 ... HelloDMCEntity(JisData)->Global: {"HelloMessage":"Hello World and DMC!"}
```